

Video Classification

Project 2

Deep Learning in Computer Vision

Gediminas Leisis
s243093@dtu.dk

Jonas Konzelmann
s251750@dtu.dk

Jakub Ondrejka
s243074@dtu.dk

Rian Gonzenbach
s251763@dtu.dk

1. Introduction

Video classification is a central problem in computer vision that aims to recognize actions or events from sequences of frames. Unlike static image recognition, it requires models to capture both spatial and temporal information. In this project we explored several convolution-based approaches for action recognition on a 10-class subset of the UCF-101 dataset. We implemented and compared (1) a per-frame aggregation baseline, (2) Early Fusion, (3) Late Fusion, and (4) 3D CNN architectures. All models were trained for 10 epochs with frames resized to 128×128 pixels and a batch size of 8. Performance was evaluated using predefined train/validation/test splits, followed by a study on information leakage between these splits and finally an experiment with dual-stream networks incorporating optical flow.

2. Architectures

2.1. Video Classification using 2D and 3D CNNs

We implemented and explored several models for video classification:

- **Aggregation of Per-Frame Models**

We started with the aggregation of per-frame (single frame) models which is the easiest to implement and usually performs strongly. In this model each frame is processed independently by a 2D CNN consisting of three convolutional blocks with 3x3 kernels, batch normalization, ReLU activation, and 2x2 max-pooling layers that reduce dimensions (128x128 to 64x64 to 32x32 to 16x16). The CNN extracts spatial features from each frame and outputs logits, which are then averaged across all frames to obtain a single prediction.

- **Late Fusion**

Afterwards, we implemented the Late Fusion. This approach uses two separate CNN streams (RGB and grayscale frames). Each stream has the same 2D CNN architecture as the per-frame model, with three con-

volutional blocks (3x3 kernels, batch normalization, ReLU, and 2x2 max-pooling) and processes its input independently to extract features and produce per-frame predictions. After processing, the outputs from the two networks are averaged to obtain the final video prediction.

- **Early Fusion**

The Early Fusion model differs from Late Fusion because RGB and grayscale frames are combined before entering the CNN. The combined input is then processed by a single 2D CNN with the same architecture as the previous models (three convolutional blocks, 3x3 kernels, batch norm, ReLU, and 2x2 max-pooling).

- **3D CNN**

Finally, we implemented a 3D CNN, which includes temporal dimension. The architecture includes three convolutional blocks, each followed by 3D batch normalization, ReLU, 3D convolutional kernels (3x3x3) that operate across width, height and time. Pooling layers were also applied for temporal dimensions, gradually reducing the time dimension (from 10 to 5 to 2 frames).

All models were trained, validated and tested on provided ucf-101 dataset with pre-split data (train/val/test). We trained to models using all 10 frames, on 10 epochs and used the optimal dropout of 0.5, batch size of 8 and image size of 128 x 128 pixels.

Overall, all models in figure 1 demonstrate stable convergence within 10 epochs, the gap between training and validation curves suggest mild overfitting. The Early Fusion model quickly reached 80% validation accuracy and then stabilized, indicating fast learning but limited improvement. In contrast, the 3D CNN trained more slowly but maintained the smallest gap between the curves, suggesting better generalization. The Late Fusion model achieved the highest validation accuracy.

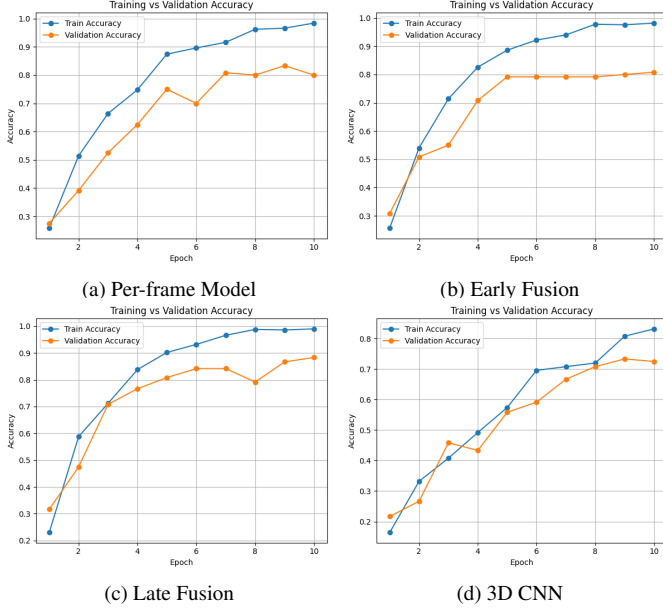


Figure 1. Training and validation accuracy curves for all models, trained for 10 epochs.

Table 1. Test accuracy for all models.

Model	Test Accuracy (%)
Per-frame aggregation	78.3
Early Fusion	85.0
Late Fusion	90.8
3D CNN	80.8

The Late Fusion model achieved the highest test accuracy of 90.8%, followed by Early Fusion model with 85.0%. The 3D CNN reached 80.8%, although we expected it to perform better since it includes the temporal dimension. The per-frame model achieved a test accuracy of 78.3% which is good for a simple baseline model.

2.2. Information leakage between train/val/test splits

When retraining the Late Fusion model on the no-leakage version of the UCF-101 subset, the performance dropped compared with the Kaggle version used in Task 4.1. While the original dataset contained overlapping subjects across training, validation, and test sets, the new split ensured that no individual appeared in more than one subset. As expected, the removal of this caused a sharp decrease in validation accuracy and a visible gap between training and validation curves. The network quickly reached nearly perfect training accuracy while the validation accuracy plateaued around 30-40 % (figure 2), indicating that the model was fitting specific individual performers rather than learning action specific features.

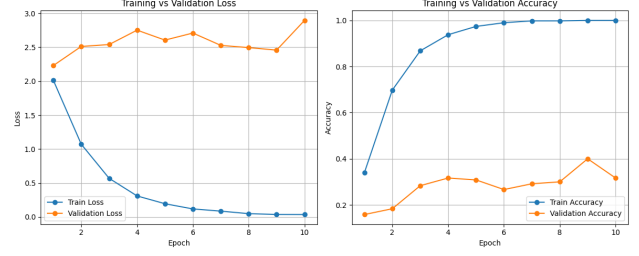


Figure 2. Training and validation accuracy curves for late fusion model, trained for 10 epochs, on ucf101noleakage dataset.

2.3. Dual-stream networks

We implemented the Dual-stream network that is shown in 3 in a straightforward way. The data loader class was also adjusted in a way that it can process both optical flow frames and image frame at the same time, which is forwarded to the Dual-stream network. The optical flow was pre calculated using the RAFT model.

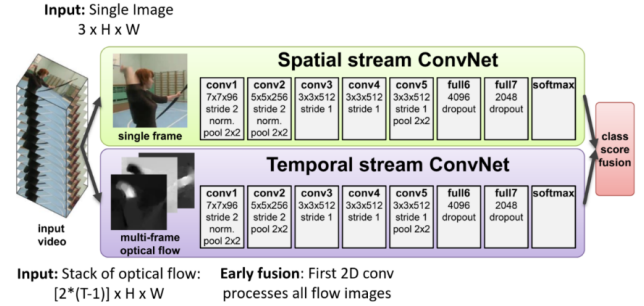


Figure 3. Two-stream architecture for video classification, Simonyan and An- drew Zisserman, NeurIPS 2014..

Afterwards, we proceeded to investigate the influence of different dropout rates and different numbers of optical flow frames used in one pass. We used the Adam optimizer with a learning rate of 0.0001, 20 epochs of training and a batch size of 8.

	1 frame	5 frames	9 frames
Dropout 0.5	40.0%	48.3%	50.0%
Dropout 0.9	30.0%	45.0%	44.2%

Table 2. Dual-stream performance tested on different dropouts and different number of used optical frames

As 2 shows, the best performance was achieved with a dropout rate of 0.5 while using all of the 9 optical flow frames. Note that the videos are 10 frames long and the optical flow is calculated between two frames, so therefore we only have 9 optical flow frames.

We then tested different batch sizes in the same setting as before, using a dropout rate of 0.5 and 9 optical flow frames.

Batch size	8	16	32	64
Performance	49.2%	59.2%	53.3%	55.0%

Table 3. Dual-stream performance tested on different batch sizes

In 3 we can see that a batch size of 16 gave the best results.

3. Conclusion

Across all tested architectures, the Late Fusion model achieved the best overall results on the original dataset, reaching $\approx 91\%$ test accuracy by combining RGB and grayscale streams. However, when retrained on leakage-free splits, accuracy dropped to $\approx 35\%$, exposing strong overfitting to subject identity rather than true motion or pose patterns. This highlights how data leakage can produce misleadingly high scores and emphasizes the importance of subject-independent splitting in video datasets. The subsequent dual-stream experiments with optical flow partially mitigated this issue by providing explicit motion information, improving generalization over purely appearance-based models.

4. Ai tools

In this project, we used Chat GPT as well as Copilot in VS Code to assist with coding, debugging, and improving the written work.

References